

STAT3009 / MACHINE LEARNING

Machine Learning I: Models and Estimators

From prediction rules to reusable code

Ben Dai

CUHK · Department of Statistics and Data Science



WE CAN ALREADY PREDICT

global mean • user mean • item mean

How do we move from hand-designed averages to a general way of learning prediction rules from data?

BEFORE

Choose a formula

Then calculate its averages from the training triples.

→

TODAY

Choose a model family

Then learn its parameters from examples.

→

THE REAL CHALLENGE

Generalize

Work well on ratings the model has never seen.



ML I • A language for prediction

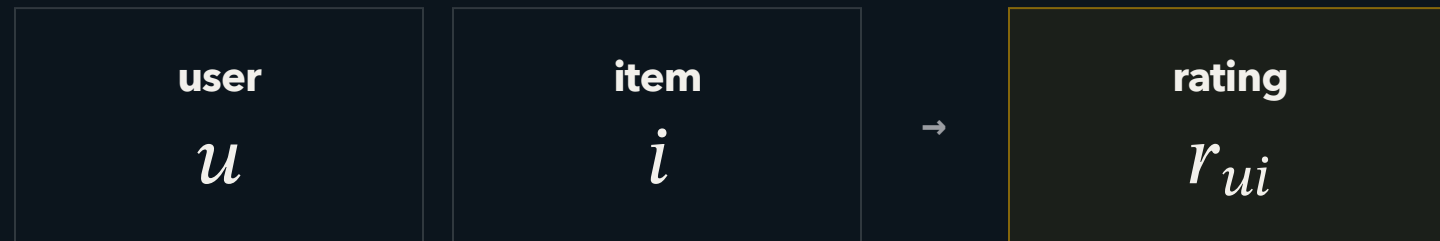
Start with X and y . Use four questions to understand a prediction model and how it learns.



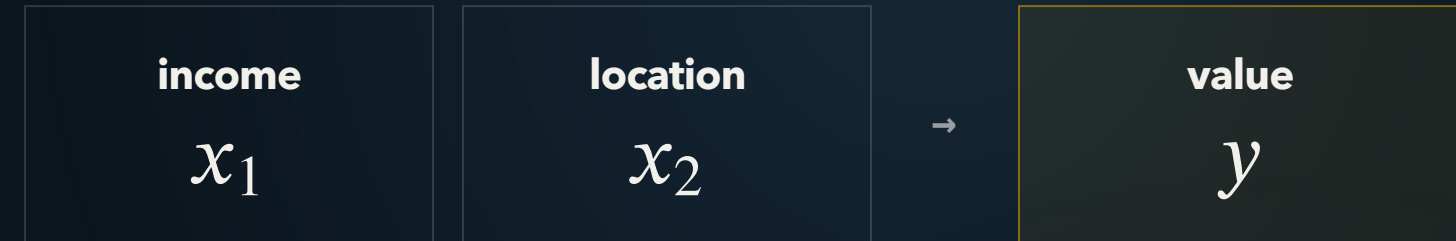
Rating prediction is supervised learning

4

RECOMMENDER SYSTEM



HOUSING REGRESSION · EXAMPLE INPUTS



Supervised learning: learn a mapping from inputs to a known numerical target, then apply it to new inputs.



One row becomes one learning example

5

RATING DATA

user	item	rating
1960	670	4
1346	152	4
785	1741	4

X contains the inputs; y contains the answer.

ONE EXAMPLE

$$x_1 = (1960, 670)$$

the user-item pair

paired with

$$y_1 = 4$$

the observed rating

The row number is not a feature. It only identifies an observation.



Four questions decode every estimator

6

1 · MODEL

$$f_{\theta,h}(x)$$

What family of prediction rules can this estimator represent?

2 · LEARNED PARAMETERS

$$\theta$$

What quantities will `fit` learn from the training data?

3 · HYPERPARAMETERS

$$h$$

What choices are fixed before `fit` and control the model or fitting rule?

4 · LOSS / CRITERION

$$L(y, \hat{y})$$

What does the fitting procedure regard as a better solution?

BEFORE FIT

choose settings

$$h$$



DURING FIT

learn from training data

$$\hat{\theta}$$

guided by the loss or fitting criterion



AFTER FIT

predict new rows

$$\hat{y} = f_{\hat{\theta},h}(x_{\text{new}})$$



ML principles through California housing

Use one dataset to understand a model family, training loss, and evaluation on new examples.



California housing: the data for our model

8

ROWS

20,640

California block groups

FEATURES

8

demographic and geographic measurements

TARGET

MedHouseVal

median house value in units of \$100,000

MedInc HouseAge AveRooms AveBedrms Population AveOccup Latitude Longitude

X_train.shape (13828, 8) y_train.shape (13828,)

Source: [scikit-learn California housing dataset documentation](#).



Linear regression combines the features

$$f_{\theta}(x) = \theta_0 + \sum_{j=1}^8 \theta_j x_j$$

 θ_0 **intercept**

the model's reference level

 θ_j **coefficient**

how prediction changes with feature j , holding others fixed

 $\hat{y}$ **prediction**

a weighted sum calculated for one row

Here each row has eight housing features. A choice of nine numbers $\theta_0, \dots, \theta_8$ gives one prediction rule.



ONE TRAINING BLOCK GROUP

$$e_k(\theta) = y_k - f_\theta(x_k), \quad \ell_k(\theta) = e_k(\theta)^2$$

ALL TRAINING BLOCK GROUPS

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^9} \frac{1}{n} \sum_{k=1}^n (y_k - f_\theta(x_k))^2$$

minimum is the smallest training loss. **arg minimum** is the coefficient vector that achieves it.

Training chooses the intercept and eight coefficients that minimize the squared errors on the training rows.



TRAINING SET

How should the model fit?

- X_{train} and y_{train} are available
- estimates parameters
- may be reused during fitting

TEST INPUT AND LABEL

What may the model see?

- X_{test} is available for prediction
- y_{test} does not estimate parameters
- y_{test} is opened only for final evaluation

Generalization is performance on new examples drawn from the task we care about.



From model to scikit-learn estimator

First understand the class and its `fit/predict` contract. Then use `LinearRegression` on the housing data.



One contract for every prediction model

CONSTRUCT

```
model = Estimator(...)
```

Choose the model and its settings.

```
model.fit(X, y)
```

LEARN

fitted estimator

Store patterns learned from the training data.

```
model.predict(X_new)
```

APPLY

prediction vector

Return one prediction for each new row.

```
LinearRegression()
```

```
KNeighborsRegressor()
```

```
YourRegressor()
```

can share the same interface

The contract specifies **how we use a model**. Each estimator supplies its own learning and prediction rules.



Class, estimator, and fitted estimator

14

PYTHON CLASS

LinearRegression

`__init__(settings)`

`fit(X, y)`

`predict(X_new)`

Defines settings and methods; `fit` later stores learned attributes.

call the class

creates

ESTIMATOR INSTANCE

```
model = LinearRegression(  
    fit_intercept=True)
```

available now

`fit_intercept`

not learned yet

`coef_`

A configured Python object, ready to receive data.

`model.fit(X, y)`

updates

FITTED ESTIMATOR

model

configuration

`fit_intercept`

learned from data

`coef_`

learned from data

`intercept_`

The same object now contains fitted state.

`model.fit(X, y)` is `model`

`True`

`fit` conventionally returns `self`.



Read LinearRegression with the four questions

15

MODEL

$$f_{\theta}(x) = \theta_0 + \sum_{j=1}^8 \theta_j x_j$$

One intercept and eight feature coefficients.

LEARNED PARAMETERS

$$\hat{\theta}_0, \dots, \hat{\theta}_8$$

Stored after `fit` as `intercept_` and `coef_`.

HYPERPARAMETER

`fit_intercept=True`

Chosen before fitting; controls whether the model includes an intercept.

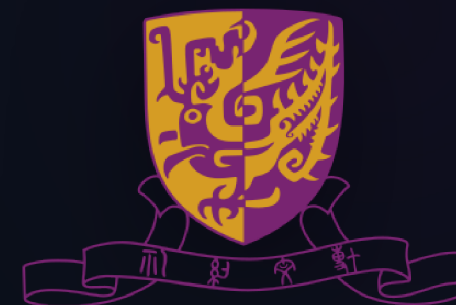
LOSS / CRITERION

$$\sum_{k=1}^n (y_k - f_{\theta}(x_k))^2$$

Ordinary least squares uses the training block groups.

THE FITTED RULE

choose `fit_intercept` → fit nine values from the training rows → predict new house values



Fit and predict with California housing

16

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression(fit_intercept=True)
model.fit(X_train, y_train)
```

```
model.coef_      # eight learned coefficients
model.intercept_ # learned intercept
```

```
y_pred = model.predict(X_test)
```

`fit_intercept`

chosen before fitting

Controls whether the model includes θ_0 .

`coef_, intercept_`

learned during fit

The trailing underscore marks fitted attributes.

`y_pred`

one value per test row

`predict` sees eight features and no target.

`model` is one object: `fit` stores the learned values; `predict` applies them to new block groups.



Many errors become one RMSE

Illustrative test rows; house values in units of \$100,000



`root_mean_squared_error(y_test, y_pred)` calculates RMSE from the true and predicted test values.



Recommender baselines: same template, new models

Now X contains user-item pairs and y contains ratings. Answer the four questions before coding each mean rule.



GlobalMeanRS: specify before coding

19

MODEL

$$f_c(u, i) = c$$

The family contains every constant prediction rule.

HYPERPARAMETERS

none

No setting is chosen before fitting.

LOSS / CRITERION

$$J(c) = \sum_{j=1}^n (r_j - c)^2$$

$r_1, \dots, r_n$ are the training ratings. Compare constants by total squared error.

FIT · OPTIMIZE

MINIMIZE THE LOSS

$$\hat{\mu} = \arg \min_{c \in \mathbb{R}} J(c)$$



THE FITTED VALUE

$$\hat{\mu} = \frac{1}{n} \sum_{j=1}^n r_j$$

LEARNED PARAMETER AFTER `fit`

$$\hat{\mu}$$

stored as `global_mean_` · `predict` returns it once per new row



Now implement GlobalMeanRS

20

```
import numpy as np
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.utils.validation import check_is_fitted
```

```
class GlobalMeanRS(RegressorMixin, BaseEstimator):
    def fit(self, X, y):
        self.global_mean_ = np.asarray(y).mean()
        return self

    def predict(self, X):
        check_is_fitted(self, "global_mean_")
        return np.full(len(X), self.global_mean_)
```

FIT

one learned value

`global_mean_` comes from `y`.

PREDICT

one value per row

`X` determines only how many predictions to return.

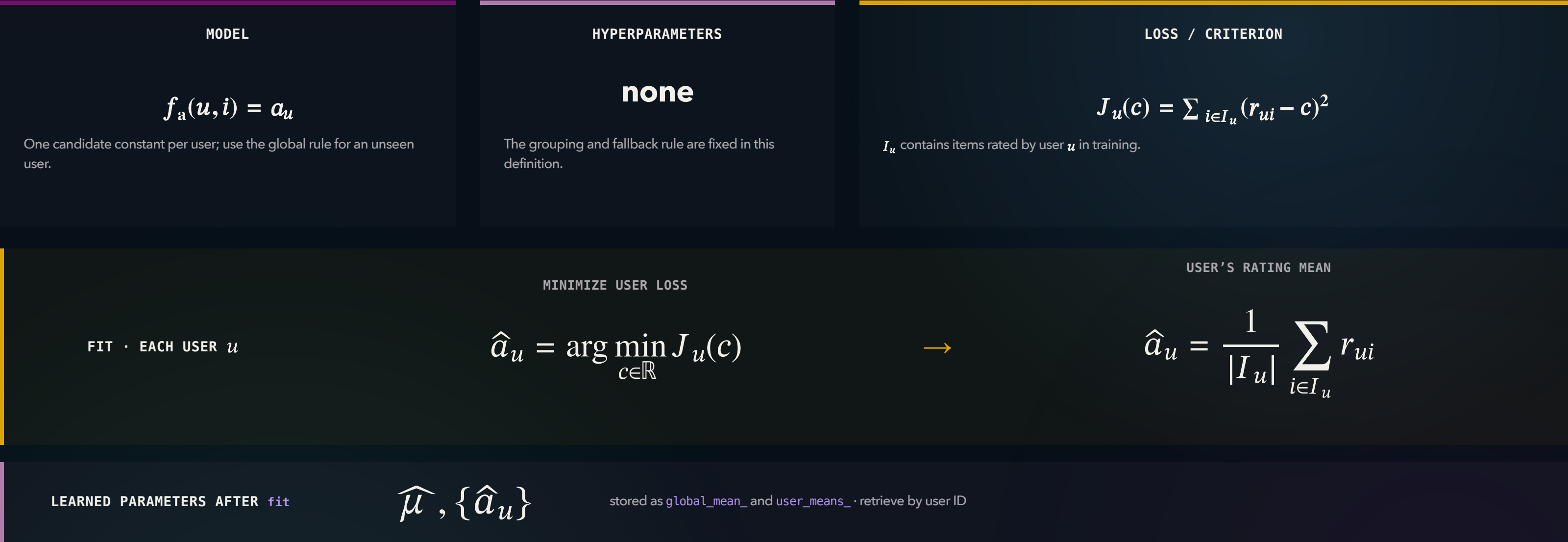
CHECK

fit must happen first

Otherwise scikit-learn raises `NotFittedError`.



UserMeanRS: specify before coding



User mean learns a lookup vector

22

GROUP TRAINING RATINGS BY USER

u_0	4	2	3	mean = 3.0
u_1	5	4	3	mean = 4.0
u_2	2			mean = 2.0

store

FITTED ATTRIBUTES

```
global_mean_ = 3.3
```

```
user ID      0    1    2  
user_means_ = [3.0, 4.0, 2.0]
```

Only a user absent from training uses `global_mean_`.

Every observed user receives a mean. The global mean handles users absent from training.



Implement UserMeanRS: fit

23

```
class UserMeanRS(RegressorMixin, BaseEstimator):
    def fit(self, X, y):
        X, y = np.asarray(X), np.asarray(y)
        users = X[:, 0].astype(int)
        self.global_mean_ = y.mean()
        self.user_means_ = np.full(
            users.max() + 1, self.global_mean_
        )

        for user in set(users):
            ratings = y[users == user]
            self.user_means_[user] = ratings.mean()
        return self
```

```
global_mean_
```

fallback learned by fit

```
user_means_
```

NumPy lookup vector learned by fit

```
users == user
```

select that user's ratings

Input: a two-column array with nonnegative user ID in column 0.



Complete UserMeanRS: predict

24

```
def predict(self, X):
    check_is_fitted(
        self, ["global_mean_", "user_means_"]
    )
    users = np.asarray(X[:, 0]).astype(int)
    y_pred = np.full(len(users), self.global_mean_)

    valid = (users >= 0) & (users < len(self.user_means_))
    y_pred[valid] = self.user_means_[users[valid]]
    return y_pred
```

`check_is_fitted`

require fit first

`y_pred`

start every row at the global mean

`valid`

which user IDs have a safe lookup index?

`y_pred[valid]`

replace fallback with learned user means

Add this method inside the `UserMeanRS` class from the previous slide.



ItemMeanRS: transfer the same template

MODEL

$$f_b(u, i) = b_i$$

One candidate constant per item; use the global rule for an unseen item.

HYPERPARAMETERS

none

The grouping and fallback rule are fixed in this definition.

LOSS / CRITERION

$$J_i(c) = \sum_{u \in U_i} (r_{ui} - c)^2$$

U_i contains users who rated item i in training.

MINIMIZE ITEM LOSS

FIT · EACH ITEM i

$$\hat{b}_i = \arg \min_{c \in \mathbb{R}} J_i(c)$$



ITEM'S RATING MEAN

$$\hat{b}_i = \frac{1}{|U_i|} \sum_{u \in U_i} r_{ui}$$

LEARNED PARAMETERS AFTER `fit`

$$\hat{\mu}, \{\hat{b}_i\}$$

stored as `global_mean_` and `item_means_` · retrieve by item ID



One workflow can use every estimator

```
models = [  
    GlobalMeanRS(),  
    UserMeanRS(),  
]  
  
for model in models:  
    fitted_model = model.fit(X_train, y_train)  
    y_pred = fitted_model.predict(X_new)  
  
    print(type(model).__name__, y_pred.shape)
```

OUTPUT · m = NUMBER OF ROWS IN X_{new}

GlobalMeanRS	(m ,)
UserMeanRS	(m ,)

Same inputs	$X_{train}, y_{train}, X_{new}$
Same methods	fit and predict
Same output contract	one prediction per new row

Model-specific logic stays inside each class. Workflow code depends only on the shared estimator interface.



One interface—and one way to read every estimator

27

ASK FOUR QUESTIONS

`model`

`learned parameters`

`hyperparameters`

`loss / criterion`

then use

`fit(X, y)`

`predict(X_new)`

next lecture

ML II

Which fitted estimator will generalize?

Holdout validation and cross-validation compare candidates without using the final test labels.

Exit ticket: for `ItemMeanRS`, name the learned attribute and explain how `predict` handles an unseen item.

